Chapter 11: Classes and Constructors

ENG3091

Khiem Nguyen

Contents

1 Member access **2**
- 1.1 Public and private . . . 2
- 1.2 An analogy . . . 2
- 1.3 Access levels . . . 3
- 1.4 Members of a `struct` are public by default . . . 3
- 1.5 Members of a `class` are private by default . . . 4
- 1.6 Setting access levels with access specifiers . . . 4
- 1.7 Access levels at a glance . . . 5
- 1.8 Access is per class, not per object . . . 5
- 1.9 Naming private member variables . . . 6
- 1.10 What real libraries do . . . 6

2 A detour: aggregates and member selection **6**
- 2.1 Aggregates . . . 6
- 2.2 Member selection through a reference . . . 7
- 2.3 Member selection through a pointer: what's wrong? . . . 8
- 2.4 Fix 1: dereference first . . . 9
- 2.5 Fix 2: the arrow operator `->` . . . 9

3 Constructors **10**
- 3.1 Why do we need constructors? . . . 10
- 3.2 ...and that's a good thing . . . 10
- 3.3 Constructors . . . 10
- 3.4 Writing a constructor . . . 11
- 3.5 A first constructor . . . 11

4 Member initializer lists **12**
- 4.1 Assigning in the body, Python style . . . 12
- 4.2 `this`: C++'s `self` . . . 13
- 4.3 The member initializer list . . . 13
- 4.4 Why is it better than assignment? . . . 14
- 4.5 Initializer list or assignment? . . . 14
- 4.6 Formatting the member initializer list . . . 15
- 4.7 Which initializer wins? . . . 16
- 4.8 The constructor body . . . 16

5 Default constructors **17**
- 5.1 The default constructor . . . 17
- 5.2 Default arguments make a default constructor . . . 17
- 5.3 Overloaded constructors . . . 18
- 5.4 But only one default constructor! . . . 19

5.5 The implicit default constructor . . . 19
5.6 Writing any constructor removes it . . . 20
5.7 `= default`: "compiler, write the usual one" . . . 20

6 Copy constructors **21**
6.1 Copying objects . . . 21
6.2 How does `Fraction f_copy{f}` work? . . . 22
6.3 The copy constructor . . . 22
6.4 Writing your own copy constructor . . . 23
6.5 Using it . . . 23
6.6 Copy constructor rules . . . 24
6.7 Pass by value calls the copy constructor . . . 24
6.8 Return by value and copy elision . . . 25
6.9 `= default` and `= delete` for copying . . . 25

7 Wrapping up **26**
7.1 Summary . . . 26
7.2 Keep learning . . . 26

1 Member access

1.1 Public and private

In Chapter 10 we used `struct`s everywhere, which showed that a `struct` is almost the same as a `class`. The one difference that matters here:

- the members of a `struct` are **public** by default;
- the members of a `class` are **private** by default.

Now let's see what `public` and `private` actually mean.

1.2 An analogy



You're tired on the way home and want to sit down somewhere. Two options:

- The park, full of screaming kids. It's `public`: you **can** go in.
- The warm, quiet house with the TV. It's `private`: you **can't**.

It's not about what you want; it's about what you're allowed to do.

1.3 Access levels

Every member of a class type has an **access level**, which decides who may use it:

1. `public`: anyone;
2. `private`: only the class's own members;
3. `protected`: we'll meet this one in Chapter 14, with inheritance.

Code that isn't a member of the class, like `main()`, is called **the public**.

1.4 Members of a `struct` are public by default

Listing 1 `struct_public_by_default.cpp`

```cpp
#include <iostream>

struct Date {
    int year{}, month{}, day{}; // public by default

    void print() const { // public by default
        std::cout << year << '/' << month << '/' << day << '\n';
    }
};

int main() { // main() is not a member: it's "the public"
    Date today{2026, 2, 26};
    today.day = 42;   // allowed: day is public...
    today.month = 42; // ...so nothing stops nonsense like this
    today.year = -42;
    today.print();
    return 0;
}
```

```
-42/42/42
```

A **public member** can be used by anyone: by the class's own members, and by the public. Nothing stops this nonsense date.

1.5 Members of a `class` are private by default

Listing 2 `class_private_by_default.cpp`

```cpp
#include <iostream>

class Date { // now a class: members are private by default
    int year{}, month{}, day{};

    void print() const {
        std::cout << year << '/' << month << '/' << day << '\n';
    }
};

int main() {
    Date today{2026, 2, 26}; // error: not an aggregate any more
    today.day = 42;          // error: day is private
    today.print();           // error: print() is private
    return 0;
}
```

```
error: no matching function for call to 'Date::Date(<brace-enclosed initializer list>)'
error: 'int Date::day' is private within this context
error: 'void Date::print() const' is private within this context
```

A **private member** can only be used by other members of the same class, never by the public.

1.6 Setting access levels with access specifiers

Listing 3 `access_specifier_example.cpp`

```cpp
#include <iostream>

class Date {
  public:                // everything below is public...
    void print() const { // members may use private members
        std::cout << year << '/' << month << '/' << day << '\n';
    }

  private: // ...and everything below is private
    int year{2026};
    int month{2};
    int day{26};
};

int main() {
    Date today{};
    today.print(); // fine: print() is public
    return 0;
}
```

```
2026/2/26
```

An **access specifier**, `public:`, `private:` or `protected:`, sets the access level of every member that follows it.

The usual layout: the **public interface** first (what users of the class need), then the **private** details.

1.7 Access levels at a glance

Access level	Specifier	Members of the class	Derived classes	The public
Public	`public:`	yes	yes	yes
Protected	`protected:`	yes	yes	no
Private	`private:`	yes	no	no

Don't worry about the middle column yet: derived classes come with inheritance, in Chapter 14. Notice the triangle of "yes" and the triangle of "no".

`struct` and `class` really are interchangeable: make every member of a `class` `public` and it acts like a `struct`, and vice versa.

1.8 Access is per class, not per object

Listing 4 `access_level_per_class_basis.cpp`

```cpp
#include <iostream>
#include <string>

class Person {
  private:
    std::string name{};

  public:
    void set_name(const std::string& new_name) { name = new_name; }

    void greet(const Person& other) const {
        // We may use other.name: other is a Person too
        std::cout << name << " says hello to " << other.name << '\n';
    }
};

int main() {
    Person joe{};
    joe.set_name("Joe");
    Person kate{};
    kate.set_name("Kate");
    joe.greet(kate);
    return 0;
}
```

```
Joe says hello to Kate
```

`greet()` reads `other.name`, a private member of a *different* object. That's allowed: a member function of `Person` may use the private members of **any** `Person`.

Easy to miss, and a real headache if you don't know it!

1.9 Naming private member variables

Many programmers prefix private member variables with `m_`. Here's why:

```
void set_name(const std::string& name) {
    name = name; // which name is which? This assigns the parameter to itself!
}
```

The parameter `name` **shadows** (hides) the member `name`. Write the member as `m_name`, or `name_`, and the problem disappears: `m_name = name;`.

Why mention it at all? You'll read code like this all your career. But the `m_` prefix is declining: many big projects use a trailing underscore (`name_`) or no decoration at all. And modern C++ gives us better tools, as you'll see later today.

1.10 What real libraries do

Click the links:

- **Google**: a trailing underscore, `variable_`.
- **PyTorch** (the famous machine-learning library): no prefix, no suffix. An example, lines 68–75.
- **deal.II** (a famous finite element library): no prefix, no suffix. An example, lines 2522–2721.

Real-world code looks complicated. Well, you want a six-figure salary and you want it to look simple?

> **!** A word about GenAI
>
> I'm not against GenAI chatbots. But you can hate me for saying this: if you get ChatGPT or Gemini to write your solutions for programming courses, how do you expect to get, and keep, a job in industry? **Just think about that!**

2 A detour: aggregates and member selection

2.1 Aggregates

To understand constructors, we need one more idea, and to finish off pointers from Chapter 08.

- In C++, an **aggregate** is (roughly) a class type with **only public data members** and no user-written constructors. A plain `struct` is the typical example.
- An aggregate can be initialized directly from a list of values: **aggregate initialization**.

Listing 5 aggregate_initialization.cpp

```cpp
#include <iostream>

struct Employee {
    int id{};
    int age{42}; // a default member initializer
    double wage{};
};

int main() {
    Employee frank = {1, 32, 60000.0}; // copy-list initialization
    Employee joe{2, 28, 45000.0};      // list initialization (preferred)
    Employee khiem{};                  // every member gets its default

    std::cout << frank.wage << ' ' << joe.wage << '\n';
    std::cout << "khiem: " << khiem.age << ", " << khiem.wage << '\n';
    return 0;
}
```

```
60000 45000
khiem: 42, 0
```

> Best practice
>
> Prefer the braced form without =.

2.2 Member selection through a reference

A reference acts just like the object itself, so we select its members with . as usual:

Listing 6 `reference_selection_operator.cpp`

```cpp
#include <iostream>

struct Employee {
    int id{};
    int age{};
    double wage{};
};

void print_employee(const Employee& employee) {
    // A reference acts like the object itself: use .
    std::cout << "Id: " << employee.id << '\n';
    std::cout << "Age: " << employee.age << '\n';
    std::cout << "Wage: " << employee.wage << '\n';
}

int main() {
    Employee joe{1, 34, 65000.0};
    joe.wage = 68000.0;
    print_employee(joe);
    return 0;
}
```

```
Id: 1
Age: 34
Wage: 68000
```

2.3 Member selection through a pointer: what's wrong?

Listing 7 `pointers_to_structs_select_members_error.cpp`

```cpp
#include <iostream>

struct Employee {
    int id{};
    int age{};
    double wage{};
};

int main() {
    Employee khiem{1, 42, 4.2e20};
    Employee* ptr{&khiem};
    std::cout << ptr.id << '\n'; // error: ptr is a pointer
    return 0;
}
```

```
error: request for member 'id' in 'ptr', which is of pointer type 'Employee*'
(maybe you meant to use '->' ?)
```

`ptr` is not an `Employee`; it holds the *address* of one. (And no, my salary of 4.2×10^{20} is not the error.)

2.4 Fix 1: dereference first

Listing 8 `pointers_to_structs_select_members_deref.cpp`

```cpp
#include <iostream>

struct Employee {
    int id{};
    int age{};
    double wage{};
};

int main() {
    Employee khiem{1, 42, 4.2e20};
    Employee* ptr{&khiem};
    std::cout << (*ptr).wage << '\n'; // dereference first, then .
    return 0;
}
```

```
4.2e+20
```

Now both my salary and my code are fine. The parentheses matter: `*ptr.wage` would mean `*(ptr.wage)`.

2.5 Fix 2: the arrow operator ->

Listing 9 `pointers_to_structs_select_members_arrow.cpp`

```cpp
#include <iostream>

struct Employee {
    int id{};
    int age{};
    double wage{};
};

int main() {
    Employee khiem{1, 42, 4.2e20};
    Employee* ptr{&khiem};
    std::cout << ptr->wage << '\n'; // the same thing, with ->
    return 0;
}
```

```
4.2e+20
```

The **arrow operator**, `->`, selects a member through a pointer: `ptr->wage` means exactly `(*ptr).wage`, just easier to read and type.

> Best practice
>
> Use `.` with objects and references, and `->` with pointers.

3 Constructors

3.1 Why do we need constructors?

In Introductory Programming 2 you wrote constructors in Python, as `__init__(self, ...)`. They don't just copy values (`self.age = age`); they also **check** them: is `age < 0`? Is the `id` unique? Is the birth date a real date?

A `struct` with public members can use aggregate initialization. A class with **private** members can't:

Listing 10 `aggregate_init_private_error.cpp`

```cpp
class Foo { // not an aggregate: it has private members
    int x{}, y{};
};

int main() {
    Foo foo{6, 7}; // error: no aggregate initialization for Foo
    return 0;
}
```

```
error: no matching function for call to 'Foo::Foo(<brace-enclosed initializer list>)'
```

3.2 …and that's a good thing

Refusing aggregate initialization for classes with private members makes sense:

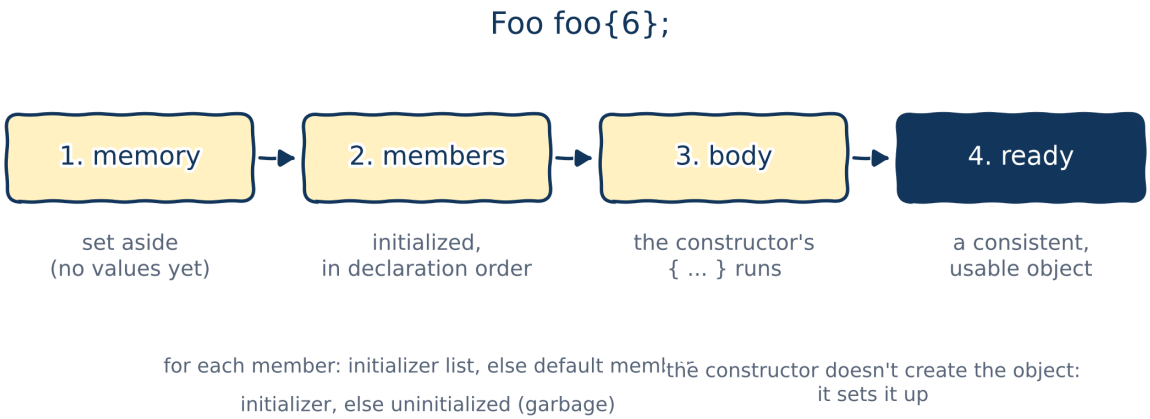
- it would require the user to know the class's **private** layout, exactly what we're trying to hide;
- if the class has an **invariant**, such as "working age is between 18 and 65", we'd be relying on every user to respect it.

So C++ classes have **constructors**, which can initialize the members *and* check the values, just like `__init__` in Python.

You can write constructors for a `struct` too; people just rarely do.

3.3 Constructors

A **constructor** is a special member function that is called **automatically** when an object of a (non-aggregate) class type is created. The compiler picks the constructor that matches the initial values given, just as it picks among overloaded functions (Chapter 05).



! Key idea

A constructor doesn't *create* the object: the memory already exists. It **sets the object up**, so that afterwards it's in a consistent, usable state.

3.4 Writing a constructor

- A constructor has **the same name as the class**, with the same capitalization.
- It has **no return type**, not even `void`.
- A class with a user-written constructor is no longer an aggregate.

```
class Student {
private:
    std::string name{};
    int age{};
    int id{};

public:
    Student(const std::string& new_name, int new_age, int new_id) {
        // set up the members here: does this remind you of __init__?
    }
};
```

3.5 A first constructor

Listing 11 `basic_constructor_example.cpp`

```
#include <iostream>

class Vector2D {
  private:
    double x{}, y{};

  public:
    Vector2D(double new_x, double new_y) { // a constructor
        x = new_x;
        y = new_y;
        std::cout << "Vector2D(" << x << ", " << y << ") constructed\n";
    }

    void print() const { std::cout << "(" << x << ", " << y << ")\n"; }
};

int main() {
    Vector2D v{4.2, 2.4}; // calls Vector2D(double, double)
    v.print();
    return 0;
}
```

```
Vector2D(4.2, 2.4) constructed
(4.2, 2.4)
```

4 Member initializer lists

4.1 Assigning in the body, Python style

Listing 12 `mimic_python_constructor_01.cpp`

```
#include <iostream>

class Foo {
  private:
    int x{}, y{};

  public:
    Foo(int x_in, int y_in) { // different names, to avoid shadowing
        x = x_in;             // assign in the body, like Python
        y = y_in;
    }

    void print() const { std::cout << "Foo(" << x << ", " << y << ")\n"; }
};

int main() {
    Foo foo{6, 7};
    foo.print();
    return 0;
}
```

```
Foo(6, 7)
```

This works, but we had to invent new names, `x_in` and `y_in`. Can we keep `x` and `y`, like `self.x = x` in Python?

4.2 `this`: C++'s `self`

Listing 13 `mimic_python_constructor_02.cpp`

```cpp
#include <iostream>

class Foo {
  private:
    int x{}, y{};

  public:
    Foo(int x, int y) { // same names as the members...
        this->x = x;    // ...so use this-> for the members
        this->y = y;
    }

    void print() const { std::cout << "Foo(" << x << ", " << y << ")\n"; }
};

int main() {
    Foo foo{6, 7};
    foo.print();
    return 0;
}
```

```
Foo(6, 7)
```

In every member function, **`this`** points at the implicit object. So `this->x` is the member; plain `x` is the parameter.

> **Coming from Python**
>
> `this` plays the role of `self`, but it's never a parameter, and you only need it when a name is ambiguous.

4.3 The member initializer list

Modern C++ has a better way than assigning in the body: the **member initializer list**.

Listing 14 `member_initializer_list.cpp`

```cpp
#include <iostream>

class Foo {
  private:
    int x{}, y{};

  public:
    Foo(int x, int y)
        : x{x}, y{y} // the member initializer list
    {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }

    void print() const { std::cout << "Foo(" << x << ", " << y << ")\n"; }
};

int main() {
    Foo foo{6, 7};
    foo.print();
    return 0;
}
```

```
Foo(6, 7) constructed
Foo(6, 7)
```

It comes after the parameter list and **starts with a colon**: each member, its value in braces, separated by commas. In `x{x}`, the outer `x` can only be a member, so there's no ambiguity.

4.4 Why is it better than assignment?

Member initializer list, `: x{x}`

- **Initialization, not assignment:** each member is created with its value, in one step.
- **Required for `const` and reference members**, which can't be left without a value and assigned later.
- **Faster** for class-type members: no default construction followed by assignment.

Assignment in the body, `this->x = x;`

1. Each member is first initialized: with its default member initializer if it has one, otherwise left with a garbage value.
2. Then the body runs, and the parameter is assigned over it.

4.5 Initializer list or assignment?

	Member initializer list `: x{x}`	Assignment in the body `this->x = x`
What happens	direct initialization	default initialization, then assignment
Efficiency	faster: no double work	slower for complex members
`const` and reference members	works	doesn't compile

	Member initializer list `: x{x}`	Assignment in the body `this->x = x`
Order	always the **declaration** order in the class	the order of the statements

> Best practice
>
> Initialize members in the member initializer list, not by assignment in the constructor body. List them in the same order as they're declared: `-Wall` warns if you don't.

4.6 Formatting the member initializer list

C++ doesn't care about spaces and line breaks, so you'll see several styles:

```
Foo(int x, int y) : x{x}, y{y} {}
```

```
Foo(int x, int y) :
    x{x},
    y{y}
{}
```

```
Foo(int x, int y)
    : x{x}
    , y{y}
{}
```

If the list is short, one line is fine. Otherwise, the third style is easiest to read and to edit.

4.7 Which initializer wins?

Listing 15 `different_initializers.cpp`

```cpp
#include <iostream>

class Foo {
  private:
    int x{};   // default member initializer: ignored, see below
    int y{42}; // default member initializer: used
    int z;     // no initializer at all: garbage!

  public:
    Foo(int x)
        : x{x} // member initializer list: wins over x's default
    {}

    void print() const {
        std::cout << "Foo(" << x << ", " << y << ", " << z << ")\n";
    }
};

int main() {
    Foo foo{6};
    foo.print();
    return 0;
}
```

```
Foo(6, 42, 0)
```

1. In the member initializer list? Use that value.
2. Else, a default member initializer? Use that.
3. Else, it's **uninitialized**. The `0` is luck: reading `z` is undefined behaviour, and g++ doesn't even warn.

4.8 The constructor body

The body is usually **empty**: the member initializer list does the initializing. Use the body for anything else the object needs, such as checking an invariant, opening a file, or allocating memory (Chapter 09).

```cpp
class Fraction {
private:
    int numerator{};
    int denominator{};

public:
    Fraction(int numerator, int denominator)
        : numerator{numerator}, denominator{denominator}
    {
        // Invariant: the denominator must not be zero.
        // The body is the place to check it, and deal with it.
    }
};
```

5 Default constructors

5.1 The default constructor

A **default constructor** is one that can be called with **no arguments**.

Listing 16 `default_constructor_example.cpp`

```cpp
#include <iostream>

class Foo {
  public:
    Foo() { std::cout << "Foo default constructed\n"; }
};

int main() {
    Foo foo1{}; // value initialization: calls Foo()
    Foo foo2;   // default initialization: also calls Foo()
    return 0;
}
```

```
Foo default constructed
Foo default constructed
```

For a class with a default constructor, **value initialization** `Foo foo1{};` and **default initialization** `Foo foo2;` both call it.

5.2 Default arguments make a default constructor

If every parameter has a default argument (Chapter 05), the constructor can be called with no arguments, so it's a default constructor too:

Listing 17 `constructor_with_default_arguments.cpp`

```cpp
#include <iostream>

class Foo {
  private:
    int x{}, y{};

  public:
    Foo(int x = 0, int y = 0) // also a default constructor
        : x{x}, y{y} {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }
};

int main() {
    Foo foo1{};     // Foo(int, int) with both defaults
    Foo foo2{6, 7}; // Foo(int, int)
    return 0;
}
```

```
Foo(0, 0) constructed
Foo(6, 7) constructed
```

5.3 Overloaded constructors

Constructors are functions, so they can be overloaded:

Listing 18 `overloaded_constructors.cpp`

```cpp
#include <iostream>

class Foo {
  private:
    int x{}, y{};

  public:
    Foo() { std::cout << "Foo constructed\n"; } // default constructor

    Foo(int x, int y) // another constructor
        : x{x}, y{y} {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }
};

int main() {
    Foo foo1{};     // calls Foo()
    Foo foo2{6, 7}; // calls Foo(int, int)
    return 0;
}
```

```
Foo constructed
Foo(6, 7) constructed
```

5.4 But only one default constructor!

Listing 19 `overloaded_default_constructors_error.cpp`

```cpp
#include <iostream>

class Foo {
  private:
    int x{}, y{};

  public:
    Foo() { std::cout << "Foo constructed\n"; } // a default constructor

    Foo(int x = 1, int y = 2) // ALSO a default constructor!
        : x{x}, y{y} {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }
};

int main() {
    Foo foo{}; // error: which default constructor?
    return 0;
}
```

```
error: call of overloaded 'Foo(<brace-enclosed initializer list>)' is ambiguous
```

Both constructors can be called with no arguments, so `Foo foo{}` is ambiguous (Chapter 05's ambiguous matches, again).

5.5 The implicit default constructor

Listing 20 `implicit_default_constructor.cpp`

```cpp
class Foo {
  private:
    int x{}, y{};
    // no constructors declared
};

int main() {
    Foo foo{}; // fine: the compiler wrote a default constructor for us
    return 0;
}
```

If a class declares **no constructors at all**, the compiler writes a public **implicit default constructor**, equivalent to:

```cpp
public:
    Foo() {}
```

no parameters, no initializer list, an empty body.

5.6 Writing any constructor removes it

Listing 21 killing_implicit_default_constructor.cpp

```cpp
class Foo {
  private:
    int x{}, y{};

  public:
    Foo(int x, int y) : x{x}, y{y} {} // removes the implicit default one
};

int main() {
    Foo foo1{1, 2}; // fine: uses Foo(int, int)
    Foo foo2{};     // error: there is no default constructor now
    return 0;
}
```

```
error: no matching function for call to 'Foo::Foo(<brace-enclosed initializer list>)'
```

Declare **any** constructor yourself, and the compiler stops writing the implicit default one. If you still need it, you must ask for it.

5.7 = default: "compiler, write the usual one"

Listing 22 explicit_default_constructor.cpp

```cpp
#include <iostream>

class Foo {
  private:
    int x{}, y{};

  public:
    Foo() = default; // "compiler, please write the usual one"

    Foo(int x, int y) : x{x}, y{y} {}

    void print() const { std::cout << "Foo(" << x << ", " << y << ")\n"; }
};

int main() {
    Foo foo1{}; // fine again
    Foo foo2{6, 7};
    foo1.print();
    foo2.print();
    return 0;
}
```

```
Foo(0, 0)
Foo(6, 7)
```

`Foo() = default;` asks the compiler to generate the default constructor it would normally have written: an **explicitly defaulted default constructor**.

6 Copy constructors

6.1 Copying objects

You've already copied objects, in your assignments:

Listing 23 `copy_constructor_vector.cpp`

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<double> original{1.1, 2.2, 3.3, 4.4};

    std::vector<double> copy1{original};  // brace initialization
    std::vector<double> copy2(original);  // direct initialization
    std::vector<double> copy3 = original; // copy initialization

    copy1[0] = 99.9; // proof that copy1 is independent:
    std::cout << "original[0]: " << original[0] << '\n';
    std::cout << "copy1[0]:    " << copy1[0] << '\n';
    return 0;
}
```

```
original[0]: 1.1
copy1[0]:    99.9
```

All three forms create a new vector as an independent copy of `original`. How does that work? Let's find out with our own class.

6.2 How does `Fraction f_copy{f}` work?

Listing 24 `copy_constructor_intro.cpp`

```cpp
#include <iostream>

class Fraction {
  private:
    int numerator{0}, denominator{1};

  public:
    Fraction(int numerator = 0, int denominator = 1)
        : numerator{numerator}, denominator{denominator} {}

    void print() const { std::cout << numerator << '/' << denominator << '\n'; }
};

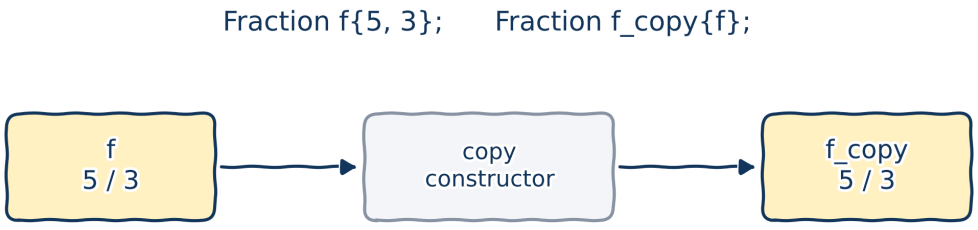
int main() {
    Fraction f{5, 3};   // calls Fraction(int, int)
    Fraction f_copy{f}; // which constructor is this?
    f.print();
    f_copy.print();
    return 0;
}
```

```
5/3
5/3
```

We never wrote a constructor taking a `Fraction`…

…but the compiler did: the **copy constructor**.

6.3 The copy constructor



two separate objects: changing one never changes the other

- A **copy constructor** initializes a new object from an existing object of the same type.
- If you don't write one, the compiler writes a public **implicit copy constructor**, which copies every member.

6.4 Writing your own copy constructor

Listing 25 `fraction.h`

```
#ifndef FRACTION_H
#define FRACTION_H

#include <iostream>

class Fraction {
  private:
    int numerator{0}, denominator{1};

  public:
    Fraction(int numerator = 0, int denominator = 1)
        : numerator{numerator}, denominator{denominator} {}

    Fraction(const Fraction& other) // our own copy constructor
        : numerator{other.numerator}, denominator{other.denominator} {
        std::cout << "Copy constructor called\n"; // just to prove it
    }

    void print() const { std::cout << numerator << '/' << denominator << '\n'; }
};

#endif
```

The copy constructor's parameter is a `const` reference to another `Fraction`; it copies each member in the member initializer list.

6.5 Using it

Listing 26 `copy_constructor_fraction.cpp`

```
#include "fraction.h"

int main() {
    Fraction f{5, 3};
    Fraction f_copy{f}; // calls Fraction(const Fraction&)
    f_copy.print();
    return 0;
}
```

```
Copy constructor called
5/3
```

The `Fraction` class now lives in `fraction.h`, so the next examples can simply `#include` it.

6.6 Copy constructor rules

Best practice

- Prefer the **implicit** copy constructor, unless you have a specific reason to write your own.
- A copy constructor should only copy, with **no side effects**: the compiler is allowed to skip calling it (next slides!).
- Its parameter should be a **`const` reference**: `Fraction(const Fraction& other)`.

Why must the parameter be a reference? If it were passed **by value**, calling the copy constructor would first need to copy the argument… using the copy constructor, which would first need to copy its argument… forever. The compiler refuses to compile that.

6.7 Pass by value calls the copy constructor

Listing 27 `copy_constructor_pass_by_value.cpp`

```cpp
#include "fraction.h"

void print_fraction(Fraction f) { // f is passed by value
    f.print();
}

int main() {
    Fraction f{5, 3};
    print_fraction(f); // f is copied into the parameter
    return 0;
}
```

```
Copy constructor called
5/3
```

Passing by value copies the argument into the parameter (Chapter 02), and for a class type that copy is made by the copy constructor. Another reason to pass class objects by `const` reference!

6.8 Return by value and copy elision

Listing 28 `copy_constructor_return_by_value.cpp`

```cpp
#include "fraction.h"

void print_fraction(Fraction f) { f.print(); } // a copy each time

Fraction make_fraction(int n, int d) {
    Fraction f{n, d};
    return f; // a copy here too? (spoiler: no)
}

int main() {
    Fraction f1{};
    print_fraction(f1);
    Fraction f2{make_fraction(1, 2)};
    print_fraction(f2);
    Fraction f3{make_fraction(4, 2)};
    print_fraction(f3);
    return 0;
}
```

```
Copy constructor called
0/1
Copy constructor called
1/2
Copy constructor called
4/2
```

In principle there are **five** copies: three into `print_fraction()`'s parameter and two for `return f;`. The output shows only three. The compiler skipped the copies in `return f;`, constructing `f` directly in the caller's object. This is **copy elision**.

6.9 `= default` and `= delete` for copying

Ask for the implicit copy constructor explicitly, or forbid copying altogether:

```cpp
Fraction(const Fraction& other) = default; // "the usual copy, please"
Fraction(const Fraction& other) = delete;  // "no copies allowed"
```

With `= delete`, any attempt to copy is a compile error:

```
error: use of deleted function 'Fraction::Fraction(const Fraction&)'
```

Try both yourself: `default_copy_constructor_using_default.cpp` and `remove_copy_constructor_using_delete.cpp` are on GitHub and Moodle.

7 Wrapping up

7.1 Summary

- `public` members can be used by anyone; `private` members only by the class's own members. Access is per **class**, not per object.
- Access specifiers set the access level; `struct` defaults to `public`, `class` to `private`.
- Select members with `.` through objects and references, and with `->` through pointers.
- A **constructor** sets up a new object. It has the class's name and no return type. Initialize members in the **member initializer list**.
- A **default constructor** takes no arguments. Writing any constructor removes the implicit one; `= default` brings it back.
- The **copy constructor** makes an object from another of the same type. It's used when passing by value, and often skipped on return (copy elision).

7.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Public and private members and access specifiers
2. Constructor member initializer lists
3. Introduction to the copy constructor
4. Class initialization and copy elision

Next up, Chapter 12: more about classes, including the `this` pointer in depth and destructors.